SR UNIVERSITY

School of Computer Science and Artificial Intelligence

Lab Assignment # 19

Name of Student : K.Saikiran
Enrollment No. : 2303A51539
Batch No. : 22

Task 1 – Python to Java Conversion
Task: Convert a Python program that performs file handling operations into an equivalent Java program using AI assistance.
Instructions:
• Prompt AI to translate a Python script that reads from and writes to a file into Java.
• Ensure proper handling of file streams, exceptions, andinput/output formatting.
• Verify that the Java program produces the same file content and console output as the Python version.
• Test both implementations with sample input files.
Expected Output:
A correctly translated and executable Java program that performs the same file handling operations as the original Python script.

Prompt:-
Convert a Python file handling program (read, write, append, delete) into Java. Ensure proper file streams, exception handling, and identical input/output behavior for both programs.

Code:- Python Implementation

```python
# ============================================================================
# TASK 1: PYTHON TO JAVA CONVERSION (FILE HANDLING)
# ============================================================================

print("\n" + "="*90)
print("TASK 1: PYTHON TO JAVA CONVERSION (FILE HANDLING OPERATIONS)")
print("="*90)

print("\n# STEP 1: ORIGINAL PYTHON CODE FOR FILE HANDLING")
print("-"*90)

python_file_handling = """
# Original Python Code - File Handling Operations
def python_file_operations():
    # Write to file
    with open('sample.txt', 'w') as file:
        file.write("Hello, World!\\n")
        file.write("This is a file handling demonstration.\\n")
        file.write("Python uses context managers for safe file operations.\\n")

    print("✓ File written successfully to 'sample.txt'")

    # Read from file
    with open('sample.txt', 'r') as file:
        content = file.read()
        print("\\nFile Content:")
        print(content)

    # Append to file
    with open('sample.txt', 'a') as file:
        file.write("This line was appended.\\n")

    print("✓ Line appended successfully to 'sample.txt'")

# Execute Python file operations
python_file_operations()
"""

print("PYTHON CODE:")
print(python_file_handling)

# Create and run Python version
print("\n# STEP 2: EXECUTING PYTHON FILE HANDLING CODE")
print("-"*90)
```

```python
try:
    # Write to file
    with open('sample.txt', 'w') as file:
        file.write("Hello, World!\n")
        file.write("This is a file handling demonstration.\n")
        file.write("Python uses context managers for safe file operations.\n")

    print("✓ File written successfully to 'sample.txt'")

    # Read from file
    with open('sample.txt', 'r') as file:
        content = file.read()
        print("\nFile Content (Python):")
        print(content)

    # Append to file
    with open('sample.txt', 'a') as file:
        file.write("This line was appended.\n")

    print("✓ Line appended successfully")

except Exception as e:
    print(f"Error in Python file handling: {e}")
```

Output:-

Python Implementation

```
PYTHON CODE:

# Original Python Code - File Handling Operations
def python_file_operations():
    # Write to file
    with open('sample.txt', 'w') as file:
        file.write("Hello, World!\n")
        file.write("This is a file handling demonstration.\n")
        file.write("Python uses context managers for safe file operations.\n")

    print("✓ File written successfully to 'sample.txt'")

    # Read from file
    with open('sample.txt', 'r') as file:
        content = file.read()
        print("\nFile Content:")
        print(content)

    # Append to file
    with open('sample.txt', 'a') as file:
        file.write("This line was appended.\n")

    print("✓ Line appended successfully to 'sample.txt'")

# Execute Python file operations
python_file_operations()


# STEP 2: EXECUTING PYTHON FILE HANDLING CODE
--------------------------------------------------------------------------------
✓ File written successfully to 'sample.txt'

File Content (Python):
Hello, World!
This is a file handling demonstration.
Python uses context managers for safe file operations.

✓ Line appended successfully
```

Code:-
Java Implementation

```
# Java equivalent code
java_file_handling = """
// EQUIVALENT JAVA CODE FOR FILE HANDLING
import java.io.*;
import java.util.*;

public class FileHandling {
    public static void main(String[] args) {
        try {
            // Write to file
            FileWriter writer = new FileWriter("sample.txt");
            writer.write("Hello, World!\\n");
            writer.write("This is a file handling demonstration.\\n");
            writer.write("Java uses FileWriter for file operations.\\n");
            writer.close();

            System.out.println("✓ File written successfully to 'sample.txt'");

            // Read from file
            BufferedReader reader = new BufferedReader(new FileReader("sample.txt"));
            String line;
            System.out.println("\\nFile Content (Java):");
            while ((line = reader.readLine()) != null) {
                System.out.println(line);
            }
            reader.close();

            // Append to file
            FileWriter appender = new FileWriter("sample.txt", true);
            appender.write("This line was appended.\\n");
            appender.close();

            System.out.println("\\n✓ Line appended successfully");

        } catch (IOException e) {
            System.out.println("Error: " + e.getMessage());
        }
    }
}
"""

print("\n# STEP 3: EQUIVALENT JAVA CODE (AI-TRANSLATED)")
print("-"*90)
print("JAVA CODE:")
```

Output:-

Java Implementation

```
JAVA CODE:

// EQUIVALENT JAVA CODE FOR FILE HANDLING
import java.io.*;
import java.util.*;

public class FileHandling {
    public static void main(String[] args) {
        try {
            // Write to file
            FileWriter writer = new FileWriter("sample.txt");
            writer.write("Hello, World!\n");
            writer.write("This is a file handling demonstration.\n");
            writer.write("Java uses FileWriter for file operations.\n");
            writer.close();

            System.out.println("✓ File written successfully to 'sample.txt'");

            // Read from file
            BufferedReader reader = new BufferedReader(new FileReader("sample.txt"));
            String line;
            System.out.println("\nFile Content (Java):");
            while ((line = reader.readLine()) != null) {
                System.out.println(line);
            }
            reader.close();

            // Append to file
            FileWriter appender = new FileWriter("sample.txt", true);
            appender.write("This line was appended.\n");
            appender.close();

            System.out.println("\n✓ Line appended successfully");

        } catch (IOException e) {
            System.out.println("Error: " + e.getMessage());
        }
    }
}
```

Justification:
Both implementations perform the same file operations with proper exception handling. The outputs remain consistent, proving functional equivalence across Python and Java.

Task 2 – C++ to Python Function Conversion
Task: Translate a C++ program that implements a sorting algorithm into Python using AI assistance.
Instructions:
• Provide AI with a C++ implementation of a sorting technique (e.g., Bubble Sort or Selection Sort).
• Ask AI to generate an equivalent Python implementation.
• Ensure logical correctness and validate results using multiple test cases.
• Confirm that the Python version maintains the same sorting behavior.
Expected Output:

A correct and executable Python program equivalent to the given C++ sorting implementation.

Prompt:-
Translate a C++ bubble sort program into Python while maintaining the same logic and input/output format. Ensure both programs produce identical sorted results.
Code:-C++ Implementation

```python
# ==================================================================
# TASK 2: C++ TO PYTHON CONVERSION (SORTING ALGORITHM)
# ==================================================================

print("\n" + "="*90)
print("TASK 2: C++ TO PYTHON CONVERSION (BUBBLE SORT ALGORITHM)")
print("="*90)

print("\n# STEP 1: ORIGINAL C++ CODE FOR BUBBLE SORT")
print("-"*90)

cpp_bubble_sort = """
// Original C++ Code - Bubble Sort Algorithm
#include <iostream>
#include <vector>
using namespace std;

void bubbleSort(vector<int>& arr) {
    int n = arr.size();

    // Traverse all array elements
    for (int i = 0; i < n - 1; i++) {
        // Last i elements are already in place
        for (int j = 0; j < n - i - 1; j++) {
            // Swap if element is greater than next element
            if (arr[j] > arr[j + 1]) {
                int temp = arr[j];
                arr[j] = arr[j + 1];
                arr[j + 1] = temp;
            }
        }
    }
}

int main() {
    vector<int> arr = {64, 34, 25, 12, 22, 11, 90};

    cout << "Original Array: ";
    for (int x : arr) cout << x << " ";
    cout << endl;

    bubbleSort(arr);

    cout << "Sorted Array: ";
    for (int x : arr) cout << x << " ";
```

Output:- C++ Implementation

```
C++ CODE:

// Original C++ Code - Bubble Sort Algorithm
#include <iostream>
#include <vector>
using namespace std;

void bubbleSort(vector<int>& arr) {
    int n = arr.size();

    // Traverse all array elements
    for (int i = 0; i < n - 1; i++) {
        // Last i elements are already in place
        for (int j = 0; j < n - i - 1; j++) {
            // Swap if element is greater than next element
            if (arr[j] > arr[j + 1]) {
                int temp = arr[j];
                arr[j] = arr[j + 1];
                arr[j + 1] = temp;
            }
        }
    }
}

int main() {
    vector<int> arr = {64, 34, 25, 12, 22, 11, 90};

    cout << "Original Array: ";
    for (int x : arr) cout << x << " ";
    cout << endl;

    bubbleSort(arr);

    cout << "Sorted Array: ";
    for (int x : arr) cout << x << " ";
    cout << endl;

    return 0;
}
```

Code:- Python Implementation

```python
print("PYTHON CODE:")
print(python_bubble_sort_code)

print("\n# STEP 3: EXECUTING PYTHON BUBBLE SORT")
print("-"*90)

def bubble_sort(arr):
    '''Bubble Sort Algorithm - Equivalent to C++'''
    n = len(arr)

    # Traverse all array elements
    for i in range(n - 1):
        # Last i elements are already in place
        for j in range(n - i - 1):
            # Swap if element is greater than next element
            if arr[j] > arr[j + 1]:
                arr[j], arr[j + 1] = arr[j + 1], arr[j]

# Test with sample data
test_cases = [
    [64, 34, 25, 12, 22, 11, 90],
    [5, 2, 8, 1, 9],
    [1, 2, 3, 4, 5]
]

for idx, arr in enumerate(test_cases, 1):
    original = arr.copy()
    bubble_sort(arr)
    print(f"Test Case {idx}:")
    print(f"  Original: {original}")
    print(f"  Sorted:   {arr} ✓")

print("\n# ANALYSIS: C++ vs PYTHON BUBBLE SORT")
print("-"*90)
print("""
```

Output:- Python Implementation

```
Test Case 1:
  Original: [64, 34, 25, 12, 22, 11, 90]
  Sorted:   [11, 12, 22, 25, 34, 64, 90] ✓
Test Case 2:
  Original: [5, 2, 8, 1, 9]
  Sorted:   [1, 2, 5, 8, 9] ✓
Test Case 3:
  Original: [1, 2, 3, 4, 5]
  Sorted:   [1, 2, 3, 4, 5] ✓
```

Justification:

The Python version follows the same sorting logic as C++. Matching outputs confirm correctness and consistency of the algorithm across both languages.

Task 3 – SQL to Pandas Query
Task: Convert a SQL query involving GROUP BY and aggregate functions into an equivalent Pandas Data Frame operation.
Instructions:
• Provide AI with a SQL query that calculates aggregate values (e.g., total sales per department).
• Prompt AI to generate equivalent Pandas code using groupby() and aggregation functions.
• Test the generated code using a sample dataset.
• Verify that both SQL and Pandas outputs match.
Expected Output:
An accurate Pandas Data Frame operation that produces results equivalent to the given SQL query with aggregation.

Prompt:-
Convert a SQL query with GROUP BY and aggregate functions into an equivalent Pandas DataFrame operation using groupby() and aggregation functions.

Code:-

```python
# ============================================================================
# TASK 3: SQL TO PANDAS CONVERSION (AGGREGATE QUERIES)
# ============================================================================

print("\n" + "="*90)
print("TASK 3: SQL TO PANDAS CONVERSION (AGGREGATE QUERIES)")
print("="*90)

print("\n# STEP 1: CREATE SAMPLE DATASET")
print("-"*90)

# Create sample dataset
sales_data = {
    'Department': ['Sales', 'IT', 'Sales', 'HR', 'IT', 'Sales', 'HR', 'IT'],
    'Employee': ['Alice', 'Bob', 'Charlie', 'Diana', 'Eve', 'Frank', 'Grace', 'Henry'],
    'Salary': [50000, 60000, 55000, 45000, 65000, 52000, 48000, 70000],
    'Bonus': [5000, 7000, 5500, 4000, 6500, 5200, 4800, 7000],
    'Year': [2024, 2024, 2024, 2024, 2024, 2023, 2023, 2023]
}

df = pd.DataFrame(sales_data)
print("\nSample Dataset (DataFrame):")
print(df.to_string(index=False))

print("\n# STEP 2: SQL QUERY (Original)")
print("-"*90)

sql_query = """
SQL Query:
SELECT
    Department,
    COUNT(*) AS EmployeeCount,
    AVG(Salary) AS AvgSalary,
    SUM(Bonus) AS TotalBonus,
    MAX(Salary) AS HighestSalary
FROM sales_data
GROUP BY Department
HAVING AVG(Salary) > 45000
ORDER BY AvgSalary DESC;
"""
print(sql_query)

print("\n# STEP 3: EQUIVALENT PANDAS CODE (AI-TRANSLATED)")
print("-"*90)
```

```
print("\n# STEP 3: EQUIVALENT PANDAS CODE (AI-TRANSLATED)")
print("-"*90)

pandas_code = """
Python (Pandas) Code:
# Equivalent to SQL query using Pandas
result = (df.groupby('Department')
    .agg({
        'Employee': 'count',
        'Salary': ['mean', 'max'],
        'Bonus': 'sum'
    })
    .rename(columns={
        'count': 'EmployeeCount',
        'mean': 'AvgSalary',
        'max': 'HighestSalary',
        'sum': 'TotalBonus'
    })
)

# Filter (HAVING clause)
result = result[result['Salary']['mean'] > 45000]

# Sort
result = result.sort_values(('Salary', 'mean'), ascending=False)
"""
print(pandas_code)

print("\n# STEP 4: EXECUTING PANDAS CONVERSION")
print("-"*90)

# Perform groupby aggregation
result = df.groupby('Department').agg({
    'Employee': 'count',
    'Salary': ['mean', 'max'],
    'Bonus': 'sum'
})

# Flatten column names
result.columns = ['EmployeeCount', 'AvgSalary', 'HighestSalary', 'TotalBonus']

# Filter (HAVING clause)
result = result[result['AvgSalary'] > 45000]
```

Output:-

```
Sample Dataset (DataFrame):
Department Employee  Salary  Bonus  Year
     Sales    Alice   50000   5000  2024
        IT      Bob   60000   7000  2024
     Sales  Charlie   55000   5500  2024
        HR    Diana   45000   4000  2024
        IT      Eve   65000   6500  2024
     Sales    Frank   52000   5200  2023
        HR    Grace   48000   4800  2023
        IT    Henry   70000   7000  2023

# STEP 2: SQL QUERY (Original)
--------------------------------------------------

SQL Query:
SELECT
    Department,
    COUNT(*) AS EmployeeCount,
    AVG(Salary) AS AvgSalary,
    SUM(Bonus) AS TotalBonus,
    MAX(Salary) AS HighestSalary
FROM sales_data
GROUP BY Department
HAVING AVG(Salary) > 45000
ORDER BY AvgSalary DESC;


# STEP 3: EQUIVALENT PANDAS CODE (AI-TRANSLATED)
--------------------------------------------------

Python (Pandas) Code:
# Equivalent to SQL query using Pandas
result = (df.groupby('Department')
    .agg({
        'Employee': 'count',
        'Salary': ['mean', 'max'],
        'Bonus': 'sum'
    })
    .rename(columns={
        'count': 'EmployeeCount',
        'mean': 'AvgSalary',
        'max': 'HighestSalary',
        'sum': 'TotalBonus'
```

Justification:

Both SQL and Pandas implementations produce identical aggregated results. This shows accurate data transformation and equivalent data processing logic.

Task 4 – Real-Time Application: Multi-Language Snippet Converter
Scenario:
Students will select a practical real-world application requiring the same logic in two different programming languages.
Example:
• Converting a REST API implementation from Python (Flask) to Java (Spring Boot).
• Translating a data processing script from Python to C++ for performance optimization.
Instructions:
• Provide AI with source code in one programming language.
• Prompt AI to convert the code into another chosen language.
• Execute and test both versions to ensure identical functionality.
• Document differences observed in syntax, libraries, and execution behavior.
Expected Output:
Functionally equivalent and executable code snippets implemented in two different programming languages, along with documented observations.

Prompt:-
Convert a Python program that calculates standard deviation into C++. Maintain the same logic for mean and variance, and optimize for performance.
Code:- Python Implementation

```python
# Task-4
# "create a Python script that calculates the standard deviat
# Python Implementation
import math
from numpy import std
from numpy import std
from py_compile import main
def calculate_standard_deviation(numbers):
    n = len(numbers)
    if n == 0:
        return 0
    mean = sum(numbers) / n
    variance = sum((x - mean) ** 2 for x in numbers) / n
    return math.sqrt(variance)
# Example usage
numbers = [10, 12, 23, 23, 16, 23, 21, 16]
std_dev = calculate_standard_deviation(numbers)
print(f"\n Standard Deviation: {std_dev}")
```

Output:- Python Implementation

```
Standard Deviation: 4.898979485566356
```

Code:-

```cpp
# C++ Implementation
#include <iostream>
#include <vector>
#include <cmath>
double calculateStandardDeviation(const std::vector<double>& numbers) {
    int n = numbers.size();
    if (n == 0) {
        return 0;
    }
    double mean = 0;
    for (double num : numbers) {
        mean += num;
    }
    mean /= n;
    double variance = 0;
    for (double num : numbers) {
        variance += (num - mean) * (num - mean);
    }
    variance /= n;
    return std::sqrt(variance);
}
int main() {
    std::vector<double> numbers = {10, 12, 23, 23, 16, 23, 21, 16};
    double stdDev = calculateStandardDeviation(numbers);
    std::cout << "Standard Deviation: " << stdDev << std::endl;
    return 0;
}
```

Output:-

```
Standard Deviation: 4.89898
```

Justification:-

Both implementations compute standard deviation using the same formula. C++ improves performance, while Python offers simplicity, highlighting language differences.

Task -5: (Basic Function Translation (Python to C++)
Translate a simple Python function that checks whether a number is even or odd into C++ using AI assistance.
Instructions:
• Provide AI with a basic Python function that determines if a number is even or odd.
• Prompt AI to generate equivalent C++ code.
• Ensure correct syntax, input handling, and output formatting.
• Compile and run the C++ program to verify it produces the same result as the Python version.
• Test with at least three different input values.
Expected Output:
A simple and executable C++ program equivalent to the given Python function, correctly identifying whether a number is even or odd for all test cases.

Prompt:-
Translate a Python function that checks whether a number is even or odd into C++. Ensure identical logic, syntax correctness, and consistent output formatting.

Code:-

```python
# Task-5
# "create a translation of a simp
# Python Implementation
from ast import main
def check_even_odd(number):
    if number % 2 == 0:
        return "Even"
    else:
        return "Odd"
# Example usage
for num in range(1, 11):
    result = check_even_odd(num)
    print(f"{num} is {result}")
```

Output:-

```
1 is Odd
2 is Even
3 is Odd
4 is Even
5 is Odd
6 is Even
7 is Odd
8 is Even
9 is Odd
10 is Even
```

Code:-

```
#include <iostream>
#include <string>
#include <vector>
#include <sstream>
#include <cmath>
std::string checkEvenOdd(int number) {
    if (number % 2 == 0) {
        return "Even";
    } else {
        return "Odd";
    }
}
int main() {
    for (int num = 1; num <= 10; num++) {
        std::string result = checkEvenOdd(num);
        std::cout << num << " is " << result << std::endl;
    }
    return 0;
}
```

Output:-

```
1 is Odd
2 is Even
3 is Odd
4 is Even
5 is Odd
6 is Even
7 is Odd
8 is Even
9 is Odd
10 is Even
```

Justification:-

Both functions use the same modulus logic to determine even/odd. Matching outputs across test cases confirm correctness and equivalence.